

CSEN 296B-2 | Week 2.1 Lab Submission

Definition of Done for AstraNotes

Student Name: Alex Shirazi

Date: April 6, 2026

Project: AstraNotes

Chosen Technical Path: Python

1. Definition of Done for AstraNotes

For my AstraNotes project, a task, feature, or artifact is done only if all of the following are true. All criteria must be satisfied unless the item is explicitly classified as a spike or experimental exploration.

1.1 Requirement Traceability

The work maps to at least one identified requirement in the Requirement Set (FR, NFR, or SPR series from Submission 2). The requirement ID is recorded in the task entry in backlog.md. If the work cannot be traced to an existing requirement, a new one is formally added before the task is marked done, or the task is removed from the backlog as out-of-scope.

AstraNotes has a defined scope — local Python note management with encryption and search. The traceability requirement prevents scope creep from being accepted as done work.

1.2 Explainability

I can explain the logic, design, and intent of every significant part of the output in plain language without referring back to the AI tool that generated it. For code, this means I can walk through the major code paths line by line. For documents, I can justify every structural decision. If I cannot explain a section, it is not done — it is a draft requiring further iteration.

The layered Python architecture (Repository, PrivacyService, ValidationLayer) involves interconnected components. Integrating code I cannot explain creates hidden dependencies and bugs that compound across sprints.

1.3 Quality and Realism Check

The output has been checked for clarity, realism, and project fit. Specifically:

- Code does not over-engineer for scale or features that are out of scope unless required by a specific NFR.
- Documentation does not promise capabilities that have not been built or tested.

- Design decisions are justifiable with the time and resources actually available in the project.

AI frequently produces over-engineered solutions. A realism check catches suggestions like adding a cache layer that are inappropriate for a local Python app with a 500-note scope (NFR-01).

1.4 Testing and Validation

For code: at least one unit test exists verifying the primary behavior of each new component before it is marked done. This maps directly to SPR-03: all core components shall have at least one automated unit test before each milestone delivery.

For documents: I have reviewed the output against its stated purpose and checked for internal contradictions, vague claims, and unsupported assertions.

An untested PrivacyService that silently fails means private notes are stored in plaintext, directly violating SPR-01. Testing is the mechanism that makes privacy enforceable.

1.5 Security, Privacy, and Governance Compliance

Any work that touches note data must satisfy the following before being marked done:

- Private notes (`is_private == True`) are encrypted via PrivacyService before any persistence operation. Raw note content is never written to disk in plaintext (SPR-01).
- Error messages do not expose note content, user data, or internal stack traces to the interface layer (SPR-02).
- Any new third-party library introduced is documented with its purpose and license in the project dependency log (SPR-04). Work that introduces an unreviewed dependency is not done.

Privacy is a first-class requirement. Marking a persistence feature as done when PrivacyService integration has not been verified is a governance failure that is easy to miss when moving fast with AI-generated code.

1.6 Work Deserves to Move Forward

Before closing any task, I ask: if this work becomes the foundation for the next sprint, will it help or hurt?

- The output is not a dead end — it either delivers user value or enables work that will.
- The output does not introduce technical debt that will need to be unwound in a future sprint.
- The output is consistent with the current architecture and will not require refactoring when the next feature is added.

If the answer is uncertain, the task is flagged for review in the next retrospective before being marked done.

2. What I Tried on AstraNotes

Applying this Definition of Done to the work produced in Weeks 1 and 2 revealed the following:

- Requirement traceability passes — the Requirement Set from Submission 2 is thorough and maps cleanly to the architecture from Submission 1.
- Explainability passes — I have reviewed and can explain all architecture decisions and requirement rationale.
- Testing is a real gap — no automated tests have been written yet. Under this DoD, code-adjacent work would be Draft / Pending Test, not Done.
- Security compliance is a real gap — the PrivacyService is defined in the architecture but not yet implemented or tested. Any persistence work is not done until PrivacyService integration is verified.

The first several implementation sprints must prioritize closing the testing and security verification gaps before any feature can legitimately move to Done.

3. Short Reflection

Writing this Definition of Done made one thing immediately clear: the most common failure mode for an AI-assisted project is confusing generated with done. AI can produce a complete-looking module in seconds, but without a traceability check, an explainability review, a unit test, and a privacy compliance pass, that module is a liability rather than an asset.

The DoD also surfaced a structural gap in my workflow: I have no explicit gate between “AI generated this” and “I verified this.” Adding that gate will slow down individual tasks but dramatically improve the reliability of work that reaches the milestone boundary. The Definition of Done is not a checklist for its own sake — it is the mechanism by which AI-assisted work earns the right to be called finished.